The Walt Disney Studios
Technology Innovation Group

Distributed Contracts
Technical Design

Introduction

The purpose of this document is to provide the technical design for implementing distributed contracts. The scope of the technical design focuses on the following use-cases:

- Browse – User browses content via Client.
- Purchase – User purchases content from Disney via Client using the Oracle and Ledger.
- Playback – User playbacks content via Client using the Ledger, Licenser and BitTorrent.

Terminology

Actors:

- *User* – Initiates purchase and playback of content.
- *Distributor* – Provides client application for user.
- *Publisher* – Provides content and issues entitlements.

Systems:

- *Client* – Retail application used to purchase and playback content. Provided by Distributor.
- *Oracle* – Distributed service to execute contract. Provided by neutral 3rd party.
- *Ledger* – Bitcoin ledger to record all transactions. Provided by neutral 3rd party.
- *Licenser* – Service to handle issue of licenses for playback. Provided by Publisher.
- *AssetPublisher* – Service to provide asset data. Provided by Publisher.
- *BitTorrent* – Distributed file-sharing network used to distribute assets. Provided by peer clients.

Use-Case: Browse

This use-case focuses on the user experience via the client application when browsing and viewing content.

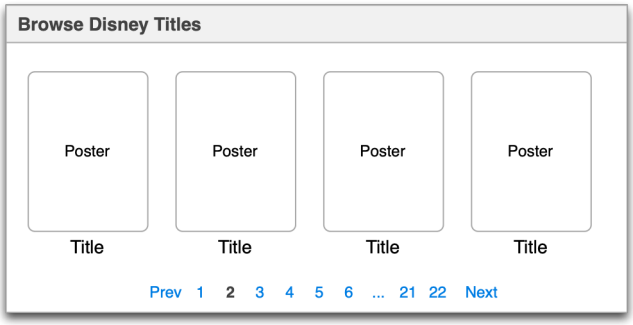

Figure 1. Browse Content Functionality

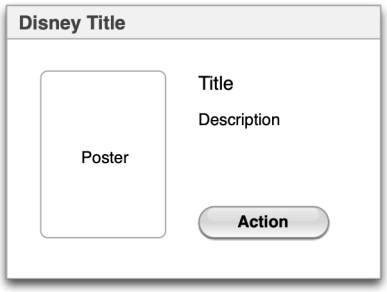

Figure 2. View Title Content Functionality

Figure 3. Playback Content Functionality

The Distributor will include the above functionality in the Client application. The Client application will be provided by the Distributor and installed locally by the User.

Use-Case: Purchase

This use-case focuses on purchasing of content by the user via the client application. This focuses on a financial transaction that produces an entitlement assigned to the User for a specific title. This transaction will be an "atomic coin swap" between bitcoin and asset coins. The steps in this use-case are as follows.

1. Client initiates contract via trusted Oracle.
2. Oracle creates a raw Transaction.
3. Oracle returns the Transaction to Client.
4. Client signs the payment input for the User.
5. Client sends updated Transaction to Oracle.
6. Oracle sends Transaction to AssetPublisher.
7. AssetPublisher signs the asset input.
8. AssetPublisher returns updated Transaction to Oracle.
9. Oracle submits Transaction to Bitcoin network for confirmation.
10. Transaction added to blockchain ledger after confirmation.

The following sequence diagram illustrates this transaction flow.

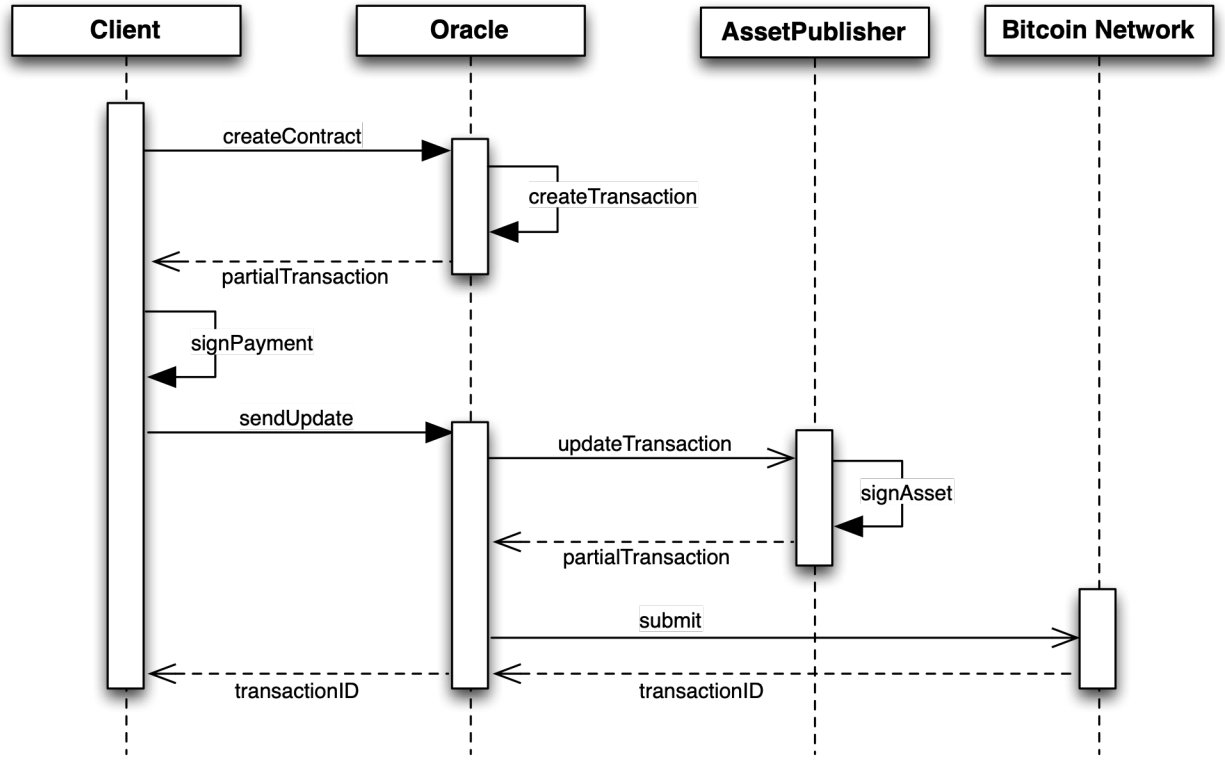

Figure 4. Sequence Diagram for Content Purchase

Method	Purpose	Return Value	Error State & Recovery
createContract	Initiates Oracle contract	Partial transaction	• Oracle unreachable: retry
createTransaction	Create transaction from contract template	Partial transaction	• Bitcoin unreachable: retry
signPayment	Sign bitcoin payment input	Signed transaction	• Invalid private key: purchase fails • Unable to sign: retry
sendUpdate	Send signed transaction to Oracle	None	• Oracle unreachable: retry
updateTransaction	Request Publisher to update transaction	Updated transaction	• AssetPublisher unreachable: retry
signAsset	Sign asset transfer input	Signed transaction	• Invalid private key: purchase fails • Unable to sign: retry
submit	Submit complete transaction to Bitcoin network	Transaction	• Invalid transaction: purchase fails

Figure 5. Method Definitions

For most error conditions, the system will attempt to retry the step. If the error persists after 1 retry, the step will throw a permanent failure and the purchase will fail with a user notification.

Use-Case: Playback

This use-case focuses on playback of content via the client application. The steps in this use-case are as follows.

1. Client requests asset lookup from Licenser.
2. Licenser verifies entitlement by looking up the User's asset Transaction in the Ledger.
3. Licenser requests proof of ownership from Client.
4. Client requests license by sending proof of ownership to Licenser.
5. Licenser issues license.
6. Client requests asset via BitTorrent

The following sequence diagram illustrates this transaction flow.

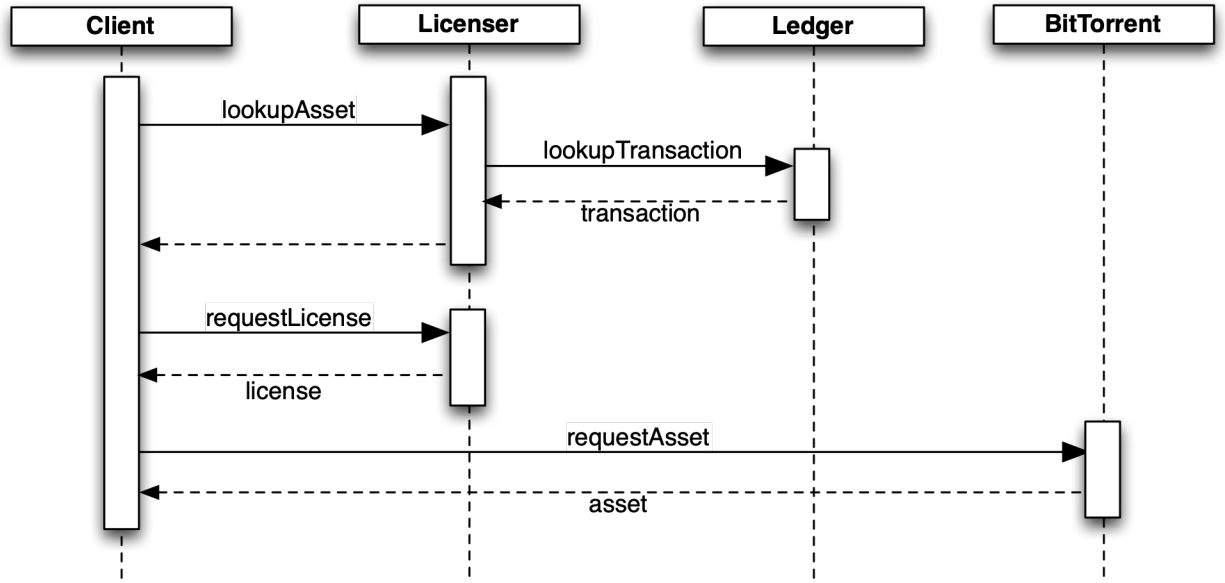

Figure 6. Sequence Diagram for Content Playback

Method	Purpose	Return Value	Error State & Recovery
lookupAsset	Lookup asset for license request	Request proof of ownership	• Licenser unreachable: retry
lookupTransaction	Lookup asset transaction in Bitcoin Ledger	Transaction	• No transaction found: license fails
requestLicense	Request license by sending proof of ownership	License	• Proof of ownership fails: license fails • Licenser unreachable: retry
requestAsset	Retrieve streamable asset from BitTorrent	Streamable asset	• Asset not found: retry

Figure 7. Method Definitions

For most error conditions, the system will attempt to retry the step. If the error persists after 1 retry, the step will throw a permanent failure and playback will fail with a user notification.

Implementation: Data Management

Key Management:

Key management is a core component of using bitcoin. In order to transact via bitcoin, the User, Distributor, and Publisher must have setup a bitcoin wallet with private and public keys prior to a purchase transaction.

Asset Management:

Asset management will be handled loosely via BitTorrent and the Client application. The Client will be built using PopcornTime as a foundation. PopcornTime currently is integrated with BitTorrent for asset delivery. The Client application will query the AssetPublisher periodically to retrieve a list of all available assets with associated metadata.

Asset Identification:

- Assets will be identified using both a bitcoin asset identifier and a magnet link
- Bitcoin asset identifier will be assigned when an asset is initially created.
- Magnet links will be generated for each torrent file based on the SHA-1 hash of the info block.
- Magnet links will be added to the metadata associated with the asset within the blockchain.

Digital Rights Management:

- Playback will be done using Widevine DRM via HTML5 EME.
- Licenser shared service will be built as a simple proxy service using Node.JS.

Proof of Ownership:

In order to provide a means of identifying the user, a proof of ownership mechanism will be used. This will be done using the Bitcoin method of proving ownership of a Bitcoin address and associated Transactions. Once the transaction with the required asset is found for a User, the Client will generate a Digital Signature using the transaction message and private key for the User. This digital signature will be provided to the Licenser as proof of ownership. The Licenser will then use the digital signature, transaction, and public key for the User to verify ownership of that Transaction. In this scenario, the public key for the User is equivalent to the Bitcoin address for the User.

Implementation: Entitlements

Entitlements will be represented as assets stored as coins within the blockchain. Entitlements will be implemented using Colored Coins, a bitcoin 2.0 platform, specifically the Open Assets Protocol (*http://www.openassets.org*) and CoinPrism (*https://www.coinprism.com*). For more information on bitcoin 2.0 platforms, see the Bitcoin 2.0 Platform Analysis section below.

Colored coins is a concept layered on top of Bitcoin, creating a new set of information about coins being exchanged. Using colored coins, bitcoins can be "colored" with specific attributes effectively turning them into tokens used to represent assets.

In order to transact using colored coins, the User and Publisher must have setup an asset-enabled bitcoin address using CoinPrism. The Publisher's wallet will be used for storing unassigned asset coins to be used in a transaction. The User's wallet will be the target destination for asset coins after a transaction.

Entitlements will be colored coin assets within CoinPrism. Transactions will be done using assets with quantity of 1 to represent a single entitlement. After a transaction, a User's asset balance should be 1 for the given asset that was transferred.

The Open Assets protocol works by including a Marker in the Transaction output via the OP_RETURN code. This Marker contains the following data.

- **Marker Tag:** tag indicating that this transaction is an Open Assets transaction
- **Version Number:** major revision number of the Open Assets Protocol
- **Asset Quantity Count:** integer representing the number of items in the asset quantity list field
- **Asset Quantity List:** list of zero or more integers representing the asset quantity of every output in order
- **Metadata Length:** length of the metadata field
- **Metadata:** metadata to be associated with transaction including asset definition information

The following methods manage entitlements via colored coins and are provided by the CoinPrism API.

- **Create Asset:**
  - Action: Create new asset for the Publisher that includes the Magnet link in asset metadata
  - Inputs: Publisher asset-enabled bitcoin address, Magnet link for BitTorrent asset
  - Outputs: Colored coin assets, Asset identifier for colored coin asset
- **Create Transaction:**
  - Action: Send 1 asset from Publisher address to User address for given Asset
  - Inputs: User address, Publisher address, Asset identifier
  - Outputs: Transaction recorded on blockchain
- **Query Assets:**
  - Action: Query to retrieve the balance of a bitcoin address including colored coin assets
  - Inputs: User address
  - Outputs: List of assets currently "owned" by User

Implementation: Transactions

Transactions will be "atomic coin swaps" between bitcoins and colored coin assets with the following data.

Transaction General Data

- **hash**: hash of the transaction
- **lock_time**: time before which the transaction cannot be included in a block
- **fees**: total fees paid to the miner in satoshis
- **amount**: total amount of the transaction in satoshis

Transaction Inputs

- *Asset: input for the asset transfer of entitlement*
  - **hash**: hash of the transaction whose output was used to create the input
  - **index**: position of the output in the transaction from which the input was created
  - **value**: value of the input in satoshis
  - **addresses**: Publisher's bitcoin address
  - **script_signature**: script used to authorize the input for use in the transaction
  - **asset_id**: asset identifier
  - **asset_quantity**: number of assets included in transaction
- *Payment: input for the bitcoin payment*
  - **hash**: hash of the transaction whose output was used to create the input
  - **index**: position of the output in the transaction from which the input was created
  - **value**: value of the input in satoshis
  - **addresses**: User's bitcoin address
  - **script_signature**: script used to authorize the input for use in the transaction

Transaction Outputs

- *Marker: output tagging transaction with Open Assets Protocol information*
  - **index**: specific position in the transaction within which the output is contained
  - **value**: unspent amount of the output in satoshis
  - **script**: script of the output using OP_RETURN
- *Asset: asset transfer of entitlement output*
  - **index**: specific position in the transaction within which the output is contained
  - **value**: unspent amount of the output in satoshis
  - **addresses**: User's bitcoin address
  - **script**: script of the output
  - **asset_id**: asset identifier
  - **asset_quantity**: number of assets included in transaction
- *Distributor Payment: bitcoin payment output for Distributor*
  - **index**: specific position in the transaction within which the output is contained
  - **value**: unspent amount of the output in satoshis
  - **addresses**: Distributor's bitcoin address
  - **script**: script of the output
- *Publisher Payment: bitcoin payment output for Publisher*
  - **index**: specific position in the transaction within which the output is contained
  - **value**: unspent amount of the output in satoshis
  - **addresses**: Publisher's bitcoin address
  - **script**: script of the output

Appendix: Bitcoin 2.0 Platform Analysis

For bitcoin 2.0, there are two platforms that were evaluated, Colored Coins and CounterParty.

Bitcoin 2.0 Platform – Feature Comparison		
Platform	**Colored Coins**	**Counterparty**
Protocol	**Open Assets**	**Counterparty Protocol**
Uses the bitcoin blockchain	Y	Y
Uses intermediate currency	N	Y
Prevents asset loss	Y	Y
Atomic bitcoin & asset swaps	Y	N
Send multiple assets in single transaction	Y	N
Send assets to multiple recipients	Y	N
Proof of authenticity	Y	N
Asset creation & transfer costs	Bitcoin fees	Bitcoin fees
Associate contract with asset	Y	N
Asset icon	Y	N
Wallet	**Coinprism**	**Counterwallet**
Send asset	Y	Y
Issue asset	Y	Y
Full modern browser support	Y	N
2-Factor-Authentication	Y	N
Send to multisig address	Y	N
Client	**Colorcore**	**Counterpartyd**
Command line	Y	Y
RPC API	Y	Y
Open source	Y	Y